RAJALAKSHMI ENGINEERING COLLEGE

Approved by AICTE | Affiliated to Anna University | Accredited by NAAC

(An Autonomous Institution, Affiliated to Anna University, Chennai)

Department of Artificial Intelligence and Data Science

V Semester B. Tech AIDS

AD23532 - Principles of Data Science

Academic Year: 2026-2027 **Batch:** 2024-2028

Ex 3b: Model Planning and Building - Regression

Roll Number: 241801187 **Date:** 06-10-2026

Dataset: Advertising (TV, Radio, Newspaper, Sales) **Tools:** Python, Pandas, NumPy, Matplotlib, Seaborn, scikit-learn

Learning Objective

To plan, build, and evaluate regression models using a real-world dataset.

Problem Statement

A marketing agency wants to help a retail company optimize its advertising budget. The company invests in three advertising channels (TV, Radio and Newspaper) and wants to understand how these expenditures influence product sales. As a Data Scientist, the task is to build and compare multiple regression models on the Advertising dataset, evaluate them with appropriate regression metrics, and communicate the findings through visualisations to support business decisions.

Task 1: Import libraries and load the Advertising dataset

In [1]:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.svm import SVR
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
sns.set_theme(style='whitegrid')

df = pd.read_csv('advertising.csv')
print("Dataset loaded successfully")
```

Out [1]:

```
Dataset loaded successfully
```

Task 2: Explore the dataset

2.1 First and last five records

In [2]:

```
print("First 5 records:"); print(df.head())
print("\nLast 5 records:"); print(df.tail())
```

Out [2]:

```
First 5 records:
      TV  Radio  Newspaper  Sales
0  230.1   37.8       69.2   22.1
1   44.5   39.3       45.1   10.4
2   17.2   45.9       69.3   12.0
3  151.5   41.3       58.5   16.5
4  180.8   10.8       58.4   17.9

Last 5 records:
        TV  Radio  Newspaper  Sales
195   38.2    3.7       13.8    7.6
196   94.2    4.9        8.1   14.0
197  177.0    9.3        6.4   14.8
198  283.6   42.0       66.2   25.5
```

```
199  232.1    8.6        8.7   18.4
```

2.2 Shape, data types, missing values and duplicates

In [3]:

```
print("Shape:", df.shape)
print("\nData types:"); print(df.dtypes)
print("\nMissing values:"); print(df.isnull().sum())
print("\nDuplicate records:", df.duplicated().sum())
```

Out [3]:

```
Shape: (200, 4)

Data types:
TV           float64
Radio        float64
Newspaper    float64
Sales        float64

Missing values:
TV           0
Radio        0
Newspaper    0
Sales        0

Duplicate records: 0
```

2.3 Summary statistics

In [4]:

```
df.describe().round(2)
```

Out [4]:

```
           TV   Radio  Newspaper   Sales
count  200.00  200.00     200.00  200.00
mean   147.04   23.26      30.55   15.13
std     85.85   14.85      21.78    5.28
min      0.70    0.00       0.30    1.60
25%     74.38    9.98      12.75   11.00
50%    149.75   22.90      25.75   16.00
75%    218.82   36.52      45.10   19.05
max    296.40   49.60     114.00   27.00
```

> **Observation:** The dataset has 200 records and 4 numerical columns, with no missing values and no duplicates, so no cleaning is needed. TV has the largest spend (mean 147.04) and Newspaper has the widest spread relative to its mean (max 114.0). Sales ranges from 1.6 to 27.0.

Task 3: Select input features and target variable

In [5]:

```
X = df[['TV', 'Radio', 'Newspaper']]   # input features
y = df['Sales']                         # target variable
print("X shape:", X.shape, "| y shape:", y.shape)
```

Out [5]:

```
X shape: (200, 3) | y shape: (200,)
```

Exploratory Data Analysis

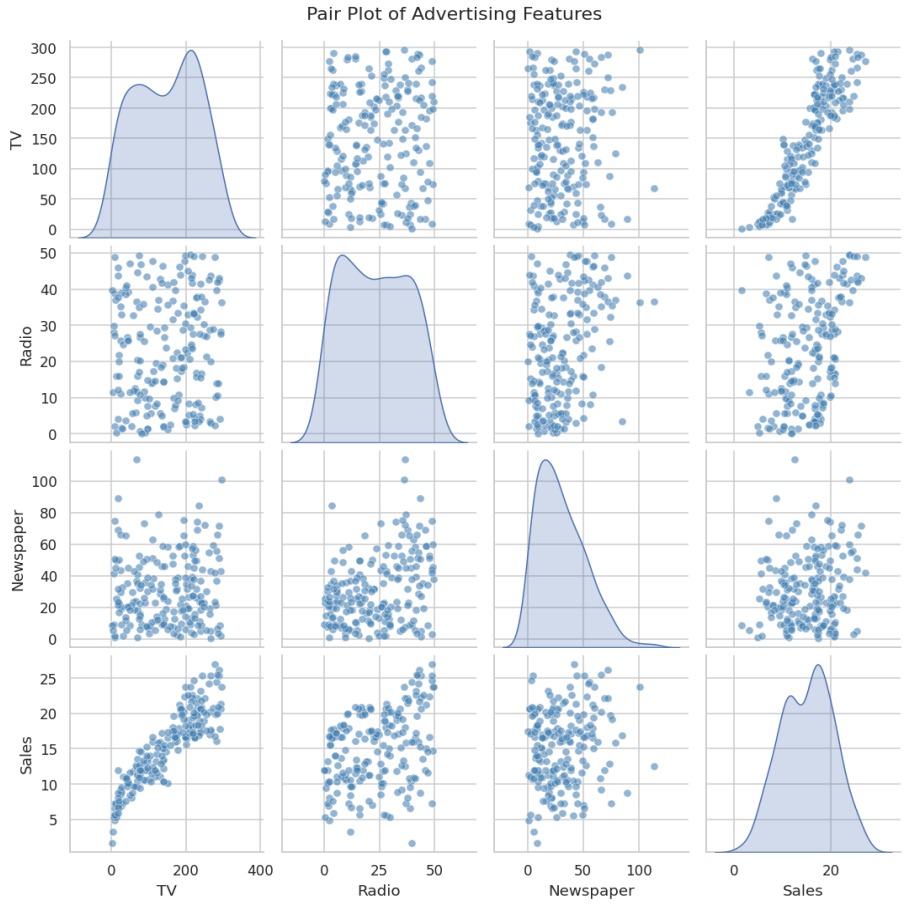
Task 4: Pair plot of the selected features

In [6]:

```
g = sns.pairplot(df, diag_kind='kde', plot_kws={'alpha': 0.6, 'color': 'steelblue'})
g.fig.suptitle('Pair Plot of Advertising Features', y=1.02)
plt.show()
```

Out [6]:

```
(pair plot shown below)
```



> **Observation:** TV shows a clear, nearly linear upward relationship with Sales. Radio shows a weaker positive trend, and Newspaper shows almost no pattern. The scatter of the TV and Radio against Sales also hints at a mild curve (sales rise more slowly at very high TV spend).

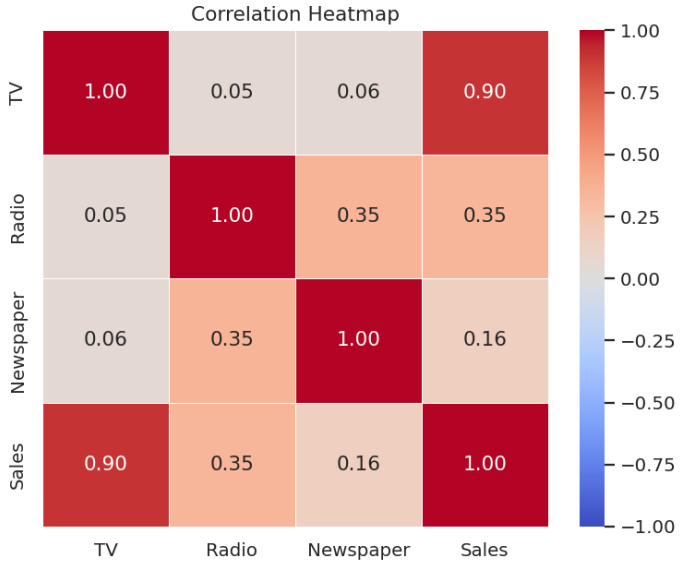
Task 5: Correlation heatmap

In [7]:

```
corr = df.corr()
plt.figure(figsize=(6, 5))
sns.heatmap(corr, annot=True, fmt='.2f', cmap='coolwarm', vmin=-1, vmax=1, center=0, linewidths=0.5)
plt.title('Correlation Heatmap')
plt.tight_layout(); plt.show()
```

Out [7]:

(heatmap shown below)



Task 6: Feature most strongly correlated with Sales

In [8]:

```
c = corr['Sales'].drop('Sales').sort_values(ascending=False)
print(c.round(3))
print("\nMost strongly correlated feature with Sales:", c.idxmax())
```

Out [8]:

```
TV           0.901
Radio        0.350
Newspaper    0.158
Name: Sales, dtype: float64

Most strongly correlated feature with Sales: TV
```

> **Observation:** TV is the most strongly correlated feature with Sales (0.901), followed by Radio (0.350) and Newspaper (0.158). The three advertising channels are almost uncorrelated with each other (TV-Radio 0.055, TV-Newspaper 0.057), so each carries independent information. Radio and Newspaper are mildly related (0.354).

Model Planning and Model Building

Task 7: Split into Training (80%) and Testing (20%)

In [9]:

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, random_state=42)
print("X_train:", X_train.shape, "| X_test:", X_test.shape)
print("y_train:", y_train.shape, "| y_test:", y_test.shape)
```

Out [9]:

```
X_train: (160, 3) | X_test: (40, 3)
y_train: (160,) | y_test: (40,)
```

Tasks 8-10: Linear, Polynomial (degree 2) and SVR (RBF kernel) models

In [10]:

```
models = {
    'Linear Regression': LinearRegression(),
    'Polynomial (deg 2)': make_pipeline(PolynomialFeatures(degree=2, include_bias=False),
                                        LinearRegression()),
    'SVR (RBF)': make_pipeline(StandardScaler(), SVR(kernel='rbf')),  # SVR needs scaled inputs
}
preds = {}
for name, model in models.items():
    model.fit(X_train, y_train)
    preds[name] = model.predict(X_test)
    print(f"{name}: trained, {len(preds[name])} test predictions")

lr = models['Linear Regression']
print("\nLinear model: Sales = %.4f + %.4f*TV + %.4f*Radio + %.4f*Newspaper"
      % (lr.intercept_, *lr.coef_))
```

Out [10]:

```
Linear Regression: trained, 40 test predictions
Polynomial (deg 2): trained, 40 test predictions
SVR (RBF): trained, 40 test predictions

Linear model: Sales = 4.7141 + 0.0545*TV + 0.1009*Radio + 0.0043*Newspaper
```

> **Observation:** The Polynomial model adds squared and interaction terms (for example TV x Radio) to a linear fit. SVR with an RBF kernel is distance-based, so the inputs are standardised inside a pipeline; the scaler is fitted only on the training data to avoid leakage. In the linear model one extra unit of Radio spend adds about 0.10 to Sales against 0.05 for TV, while Newspaper adds almost nothing (0.004).

Model Evaluation

Task 11: MAE, MSE, RMSE and R2 score

In [11]:

```
rows = []
for name, p in preds.items():
    mse = mean_squared_error(y_test, p)
    rows.append([name, mean_absolute_error(y_test, p), mse, np.sqrt(mse), r2_score(y_test, p)])
results = pd.DataFrame(rows, columns=['Model', 'MAE', 'MSE', 'RMSE', 'R2']).set_index('Model')
print(results.round(4))
```

Out [11]:

```
                      MAE     MSE    RMSE      R2
Linear Regression    1.2748  2.9078  1.7052  0.9059
Polynomial (deg 2)   0.9034  1.4425  1.2011  0.9533
SVR (RBF)            1.5554  4.4210  2.1026  0.8569
```

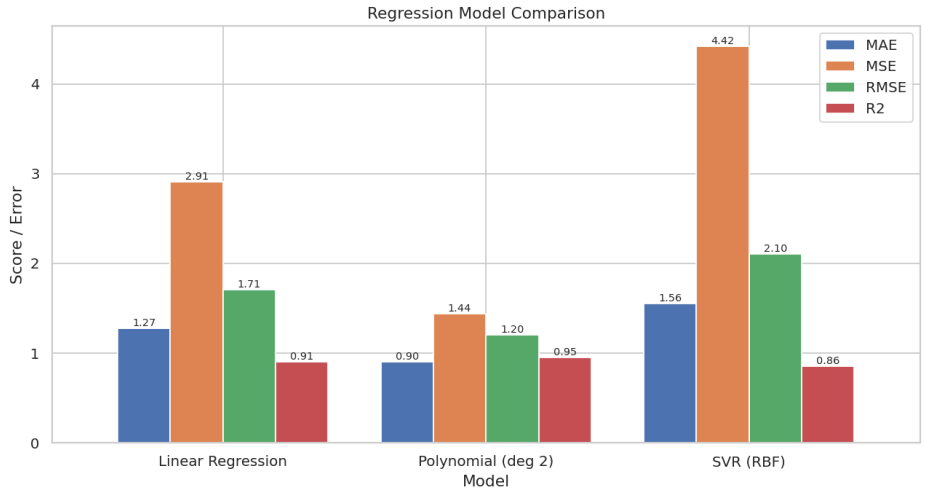
Task 12: Grouped bar chart of all models

In [12]:

```
ax = results.plot(kind='bar', figsize=(10, 5.5), rot=0, width=0.8)
for c in ax.containers:
    ax.bar_label(c, fmt='%.2f', fontsize=8)
plt.title('Regression Model Comparison'); plt.ylabel('Score / Error')
plt.tight_layout(); plt.show()
```

Out [12]:

(grouped bar chart shown below)



> **Observation:** Lower MAE, MSE and RMSE and a higher R2 indicate a better model. The Polynomial model leads on all four metrics, Linear Regression is second, and SVR with default settings is third.

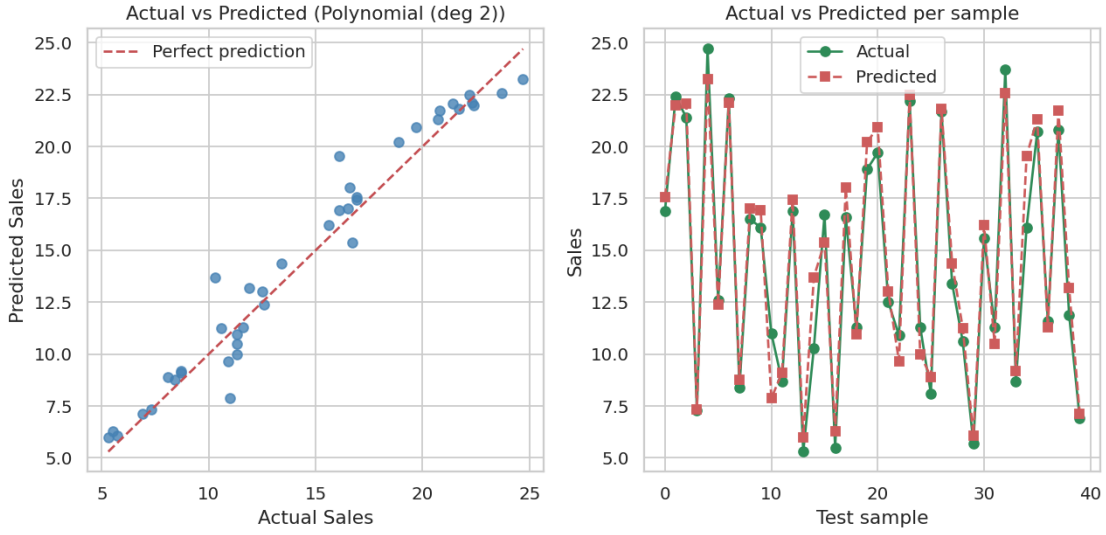
Task 13: Actual vs Predicted Sales for the best model

In [13]:

```
best = results['RMSE'].idxmin()
p = preds[best]
plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1)
plt.scatter(y_test, p, color='steelblue', alpha=0.8)
lo, hi = y_test.min(), y_test.max()
plt.plot([lo, hi], [lo, hi], 'r--', label='Perfect prediction')
plt.xlabel('Actual Sales'); plt.ylabel('Predicted Sales')
plt.title(f'Actual vs Predicted ({best})'); plt.legend()
plt.subplot(1, 2, 2)
idx = np.arange(len(y_test))
plt.plot(idx, y_test.values, 'o-', label='Actual', color='seagreen')
plt.plot(idx, p, 's--', label='Predicted', color='indianred')
plt.xlabel('Test sample'); plt.ylabel('Sales'); plt.title('Actual vs Predicted per sample'); plt.legend()
plt.tight_layout(); plt.show()
```

Out [13]:

(plot shown below)



Observation: The points of the Polynomial model lie close to the red perfect-prediction line and the predicted curve follows the actual values closely across the 40 test samples. Most errors are small (under 1.5), and three test samples (actual Sales 16.1, 10.3 and 11.0) are off by about 3 units, which accounts for much of the remaining error.

Model Interpretation

Task 14: Model with the lowest prediction error

In [14]:

```
print("Lowest RMSE :", results['RMSE'].idxmin(), "->", round(results['RMSE'].min(), 4))
print("Lowest MAE  :", results['MAE'].idxmin(), "->", round(results['MAE'].min(), 4))
print("Highest R2  :", results['R2'].idxmax(), "->", round(results['R2'].max(), 4))
```

Out [14]:

```
Lowest RMSE : Polynomial (deg 2) -> 1.2011
Lowest MAE  : Polynomial (deg 2) -> 0.9034
Highest R2  : Polynomial (deg 2) -> 0.9533
```

Observation: Polynomial Regression (degree 2) has the lowest prediction error: RMSE 1.20 against 1.71 for Linear Regression and 2.10 for SVR, and it explains 95.3% of the variance in Sales. A 5-fold cross-validation (mean R2: Polynomial 0.925, Linear 0.895, SVR 0.831) gives the same ranking, so the result does not depend on this one split. With only 40 test rows, the exact figures can shift with a different split.

Task 15: Three recommendations to optimise the advertising budget

1. **Make TV the core of the budget, but watch for diminishing returns.** TV has by far the strongest relationship with Sales (correlation 0.901). The squared TV term in the Polynomial model is negative, so very high TV spending adds less and less. Increase TV spending, and monitor the sales gained per extra unit to find the point where it levels off.
2. **Pair Radio with TV instead of treating the channels separately.** The Polynomial model, which beats the linear model, has a positive TV x Radio interaction term, so Radio spending is worth more when TV spending is also high. In the linear model Radio also gives more sales per unit (0.10) than TV (0.05). Shift part of the budget to a combined TV and Radio plan and test it with a pilot campaign.
3. **Cut back Newspaper spending.** Newspaper has the weakest link with Sales (correlation 0.158) and a near-zero effect in the linear model (0.004 per unit). Part of its small correlation comes from its link with Radio (0.354). Reduce the Newspaper budget and move the money to TV and Radio, then check that sales hold steady.

Result

Thus three regression models (Linear, Polynomial of degree 2, and SVR with an RBF kernel) were built on the Advertising dataset with an 80:20 train-test split and compared using MAE, MSE, RMSE and R2. TV was the feature most strongly correlated with Sales (0.901). Polynomial Regression gave the lowest prediction error (RMSE 1.20, R2 0.953) and was the best-performing model. Three recommendations were made to optimise the advertising budget across TV, Radio and Newspaper.